Bean Lifecycle, Scopes and Proxies

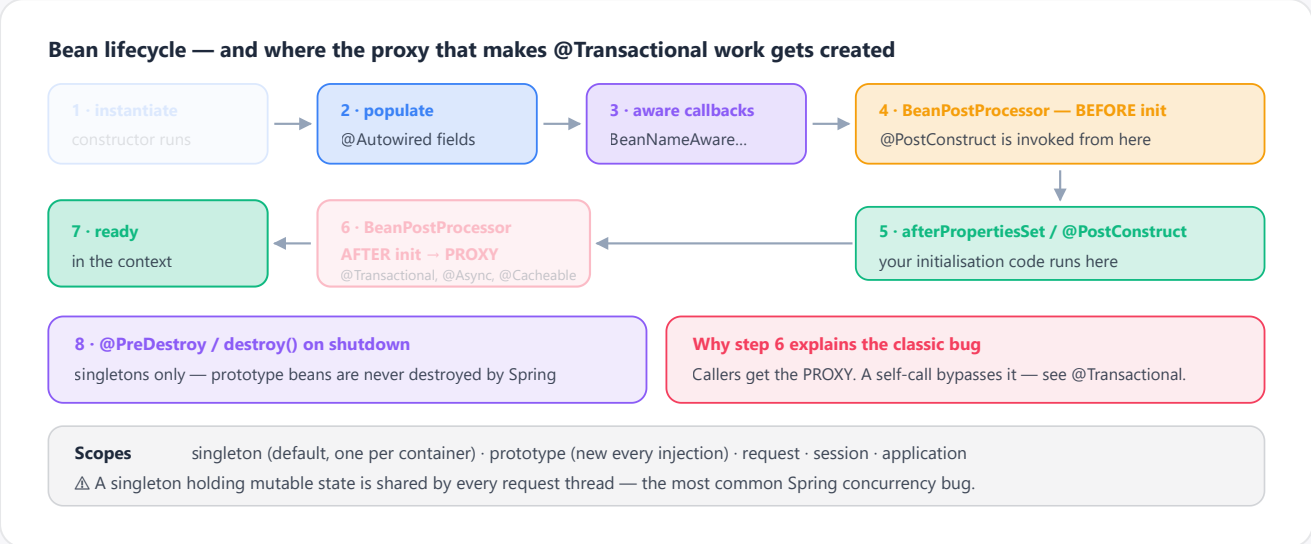
8 min read Updated 28 Jul 2026 Topic 53 of 65

Why it matters

The lifecycle looks like trivia until you meet the bug it explains: `@Transactional` silently doing nothing when a method calls another method on the same class. That behaviour is not a Spring quirk — it falls directly out of *when* and *how* Spring creates its proxies, and understanding it is what turns a memorised list of callbacks into working knowledge.

The concept, simply

A bean goes through instantiation, dependency injection, callbacks, and — critically — a post-processing step where Spring may **replace it with a proxy**.



Step 6 is the one that matters. Anything annotated @Transactional, @Async or @Cacheable is wrapped, so what other beans receive is the proxy — not your object.

How it works

Two scope facts that catch people out. **Spring does not manage prototype destruction** — it creates the bean, hands it over, and forgets it, so `@PreDestroy` never runs and any resource it holds must be closed by you. And **injecting a prototype into a singleton gives you one instance forever**, because injection happens once when the singleton is built. If you genuinely need a fresh instance per call, inject an `ObjectProvider` and ask it each time.

Proxies come in two flavours. If the bean implements an interface, Spring uses a JDK dynamic proxy that implements the same interface. Otherwise it uses CGLIB to generate a subclass. Spring Boot defaults to CGLIB for everything, which is why `final` classes and `final` methods cannot be proxied — the subclass cannot override them, so the annotation silently does nothing.

Code examples

The self-invocation problem, which is the single most-reported Spring "bug":

JAVA

Copy

```java
@Service
public class OrderService {

    public void processAll(List<Order> orders) {
        for (Order o : orders) {
            process(o);        // ✗ INTERNAL call — goes straight to "this
        }                      //   bypassing the proxy entirely.
    }

    @Transactional
    public void process(Order o) {
        // NO TRANSACTION when called from processAll above.
        // Called from another bean, it works perfectly — which is exactly
        // what makes this so confusing to debug.
    }
}

// Why: callers hold the PROXY. The proxy intercepts the call, starts a
```

```
// transaction, then delegates to the target. But "this" inside the target
// points at the target, not the proxy — so an internal call never passes through
// the interception layer.

// ✓ Fix 1 (best): move the annotated method to a separate bean, so the
//   call crosses a bean boundary and goes through the proxy.
// ✓ Fix 2: inject self via ObjectProvider and call through it.
// ✗ Non-fix: adding @Transactional to processAll as well — it changes the
//   transaction boundary but does not make the inner annotation apply.
```

Lifecycle callbacks:

JAVA Copy

```
@Component
public class ConnectionPool {

    @PostConstruct                          // after injection, before use
    public void init() { openConnections(); }

    @PreDestroy                             // on context shutdown (singleton
    public void shutdown() { closeConnections(); }
}
// @PostConstruct/@PreDestroy are the portable JSR-250 annotations and the
// idiomatic choice. InitializingBean/DisposableBean couple you to Spring;
// @Bean(initMethod, destroyMethod) suits third-party classes you cannot a

// ⚠ Constructor vs @PostConstruct: dependencies are NOT injected when the
//   constructor runs (with field injection), so initialisation that uses
//   them must go in @PostConstruct. With constructor injection this is a
//   non-issue — another point in its favour.
```

Prototype in a singleton:

JAVA Copy

```
// ✗ Injected once → the SAME "prototype" forever.
@Service
public class Handler {
    private final Task task;                    // prototype-scoped
```

```
// ✓ Ask for a new one per call.
@Service
public class Handler {
    private final ObjectProvider<Task> tasks;
    public Handler(ObjectProvider<Task> tasks) { this.tasks = tasks; }
    public void handle() { Task t = tasks.getObject(); … }   // fresh each
}
```

Under the hood

Proxy creation happens in a `BeanPostProcessor` — specifically `AbstractAutoProxyCreator` — running *after* initialisation. That ordering is exactly why `@PostConstruct` runs on the raw target and why the proxy is what everyone else receives. It also explains a subtler trap: calling an annotated method from inside `@PostConstruct` does not go through the proxy either, because the proxy does not exist yet.

Interface versus class proxies has a visible consequence. A JDK proxy only implements the interface, so injecting the concrete class fails with a confusing type error; CGLIB subclasses the class, so it cannot handle `final`. Spring Boot standardised on CGLIB to avoid the first problem, which is why the practical rule is simply: **do not make proxied classes or their annotated methods `final`, and do not annotate `private` methods** — none of them can be intercepted, and all fail silently.

The same reasoning applies to `@Async` and `@Cacheable`. They are all proxy-based, so they all share the self-invocation limitation, the `final` limitation, and the private-method limitation. Recognising that one mechanism explains all three is a strong answer.

Common mistakes

- **Self-invocation of an annotated method.** Bypasses the proxy — no transaction, no async, no caching.
- `final` or `private` **annotated methods.** Cannot be proxied; the annotation is silently ignored.

- **Expecting `@PreDestroy` on a prototype.** Spring does not manage prototype destruction.
- **Injecting a prototype into a singleton.** Resolved once — use `ObjectProvider`.
- **Initialisation logic in the constructor with field injection.** Dependencies are not set yet.
- **Calling annotated methods from `@PostConstruct`.** The proxy does not exist yet.
- **Mutable state on a singleton.** Shared across request threads.

Interview lens

Q: "Walk me through the bean lifecycle."

A: Instantiate, populate dependencies, run the aware callbacks, then `BeanPostProcessor` before-init — which is where `@PostConstruct` is invoked — then the init callbacks, then `BeanPostProcessor` after-init, which is where AOP proxies are created. Then the bean is ready. On shutdown, `@PreDestroy` runs for singletons. The proxy step is the one worth remembering, because it explains most Spring surprises.

Q: "Why does `@Transactional` not work when a method calls another method in the same class?"

A: Because the annotation is implemented by a proxy. Other beans hold the proxy, so calls from outside are intercepted and a transaction starts. An internal call goes through `this`, which is the target object, not the proxy — so nothing intercepts it. The fix is to move the annotated method into a separate bean so the call crosses a bean boundary; injecting self is possible but usually a smell.

Q: "What are the bean scopes?"

A: Singleton by default — one per `ApplicationContext`; prototype — a new instance per injection or lookup; and the web scopes request, session and application. Two catches: Spring does not destroy prototype beans, so `@PreDestroy` never runs on them, and injecting a prototype into a singleton

resolves once, giving you the same instance forever unless you use `ObjectProvider`.

Q: "JDK proxy or CGLIB?"

A: JDK dynamic proxies implement the bean's interface and require one; CGLIB generates a subclass and works without interfaces. Spring Boot defaults to CGLIB, which avoids the confusing failure where injecting a concrete class fails because the proxy only implements the interface. The consequence of CGLIB is that `final` classes and `final` or `private` methods cannot be proxied, and the annotation is silently ignored.

Q: "`@PostConstruct` or the constructor?"

A: The constructor is the right place with constructor injection, since everything is available. `@PostConstruct` matters when using field or setter injection, because dependencies are not set when the constructor runs. One caveat: the AOP proxy does not exist yet during `@PostConstruct`, so calling an annotated method from there does not go through interception.

Cheat-card

- Order: instantiate → populate → aware → **BPP before** (`@PostConstruct`) → init → **BPP after (PROXY)** → ready → `@PreDestroy`.
- Callers hold the **proxy**; `this` is the target → **self-invocation bypasses AOP**.
- Same limitation for `@Transactional`, `@Async`, `@Cacheable` — one mechanism.
- **No** `final` **or** `private` on proxied classes/methods — silently ignored.
- Scopes: singleton (default) · prototype · request · session · application.
- **Prototypes are never destroyed** by Spring; injecting one into a singleton resolves **once** → `ObjectProvider`.
- Boot defaults to **CGLIB** proxies.

QUICK CHECK — no marks, just for you.

Java Interview Kit

1. Why does @Transactional silently do nothing when a method calls another method in the same class?

- ◯ Because Spring only supports one transaction per request
- ◯ Because the annotation is implemented by a PROXY — other beans hold the proxy, but an internal call goes through 'this', which is the target object, so nothing intercepts it
- ◯ Because the transaction manager is not thread-safe
- ◯ Because internal calls run before the transaction starts

Check answer

2. You inject a prototype-scoped bean into a singleton. How many instances do you get?

- ◯ One per request thread
- ◯ It throws at startup
- ◯ One — injection happens once when the singleton is built, so you get the same instance forever
- ◯ A new one on every method call

Check answer

3. Why can a final or private method not be @Transactional?

- ◯ Because private methods have no bean definition
- ◯ They can be — there is no restriction
- ◯ Because transactions require reflection

is silently ignored

Check answer

Previous
IoC and Dependency Injection

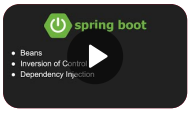
Next
Spring Boot Auto-Configuration

LEARN MORE

ARTICLES & DOCS

- Bean scopes — Spring Framework reference
  docs.spring.io
- Lifecycle callbacks — Spring Framework reference
  docs.spring.io
- AOP proxying mechanisms
  docs.spring.io

VIDEOS



Spring Core Container | Beans, IoC, DI
Mr. iTech

Hand-picked references for this lesson.